

CS-3510-B Problem Set 4

Elijah Martin McCauley

TOTAL POINTS

80 / 80

QUESTION 1

Problem 1 40 pts

1.1 a) 20 / 20

✓ - 0 pts Correct

- 3 pts Minor Error in Algorithm/ Explanation for the shortest path detection missing

- 5 pts Pseudo-code provided, but algorithm is missing.

- 5 pts Incorrect/ Missing Runtime Analysis (Ans: $(|V| + |E|)\log|V|$)

- 5 pts Incorrect/ Missing Proof of Correctness

- 2 pts Incomplete proof of correctness

- 10 pts Major Error/ Mostly Incorrect/

Suboptimal Algorithm

- 20 pts Incorrect/ Missing answer

1.2 b) 20 / 20

✓ - 0 pts Correct

- 3 pts Minor Error in Algorithm

- 5 pts Pseudo-code provided, but algorithm is missing.

- 5 pts Incorrect/ Missing Runtime Analysis

- 5 pts Incorrect/ Missing Proof of Correctness

- 10 pts Major Error/ Mostly Incorrect/

Suboptimal Algorithm

- 20 pts Incorrect/ Missing answer

- 7 pts Treated directed graph as undirected

QUESTION 2

2 Problem 2 20 / 20

✓ - 0 pts Correct

- 2 pts Minor Error in Algorithm

- 5 pts Incorrect/ Missing Runtime Analysis

- 2 pts Minor Error in Proof of Correctness

- 5 pts Incorrect/ Missing Proof of Correctness

- 10 pts Pseudo-code provided, but algorithm is missing.

- 10 pts Major Error/ Mostly Incorrect/ Suboptimal Algorithm

- 20 pts Incorrect/ Missing answer

☞ You should see it is feedback edge set because i

QUESTION 3

3 Problem 3 20 / 20

✓ - 0 pts a) Correct

![Screen_Shot_2023-02-

22_at_10.15.20_AM.png](/files/6c0efe05-a67b-4dfc-8a07-04f2674349c9)

![Screen_Shot_2023-02-

22_at_11.58.06_AM.png](/files/bebe2d42-3682-409e-b95d-62d2b7a9121b)

- 2 pts a) One to three flows incorrect

- 5 pts a) Three to six flows incorrect

- **10 pts** a) More than six flows incorrect

- **15 pts** a) Missing answer

✓ - **0 pts** b) Correct

![Screen_Shot_2023-02-

22_at_12.00.45_PM.png](/files/e0a83cac-1b48-4c8b-

ae42-208e1bee9bed)

- **1 pts** b) One or two edges incorrect

- **3 pts** b) More than two edges incorrect

- **5 pts** b) Incorrect/ Missing Answer

Problem 1

(40 points; 20 points each)

Suppose you want to find the shortest path from u to v in a graph G that *passes through* a specified vertex w . Design an algorithm that runs in $O((|V| + |E|) \log |V|)$ for each of the following settings. Justify its correctness and explain its runtime.

- a.) G is an undirected weighted graph.

Solution: In order to find the shortest path from u to v that goes through vertex w , we need to use Binary heap based Dijkstra's Algorithm which outputs the shortest path from the start to each node. We run Dijkstra's Algorithm using w as the starting point until. This will output the shortest path from w to all nodes. Then we need to flip the path from w to u to get the path from u to w . We can then concatenate the path from w to v to the end of the path from u to w to get the shortest path from u to v through w . Note: we will have to truncate the path from w to v to exclude the w , in order to get a continuous path; this makes the output $u-w-v$, not $u-w-w-v$ which would be wrong. This works because there is no shorter way to get from w to u or from w to v because that's how Dijkstra's works, so the net path has to be the shortest path. The runtime is $O((|V| + |E|) \log |V|)$ because Dijkstra's algorithm runs in $O((|V| + |E|) \log |V|)$ time.

- b.) G is a directed weighted graph, and instead of a single u and v , we have k queries (u_1, v_1) , (u_2, v_2) , $\dots$, (u_k, v_k) where for each one we want to find *only the weight* of the shortest path that passes through w (which is fixed) where $w \in G$ and $0 < k \leq |V|$.

Solution: For this problem, we would run a version of Binary Heap based Dijkstra's algorithm which returns a dictionary containing nodes and their distance to the starting node. We first run this Dijkstra's starting at w , which will get output the distances to all potential v . Then we construct a reverse graph and run Dijkstra's again with w as the starting point which will output the distances to all potential u . From there, you can simply loop through the queries and use the dictionary to find the distance from w to u and add it to the distance from w to v to find the shortest distance from u to v through w . This works because Dijkstra's can be run on a weighted graph, but would only be able to find the nodes reachable from w which would be all the options for v because we know that there is a path from u to v through w . Similarly, once we reverse the graph, all the nodes reachable by w would be the potential starting points or u . This approach would give us the distances to all the nodes in the graph that can reach or be reached by w . This method is also $O((|V| + |E|) \log |V|)$ because we are running Dijkstra's algorithm at most a constant 2 times and Dijkstra's algorithm runs in $O((|V| + |E|) \log |V|)$ time and then we are simply looking through the dictionary which is more efficient and can be ignored.

1.1 a) 20 / 20

✓ - 0 pts Correct

- 3 pts Minor Error in Algorithm/ Explanation for the shortest path detection missing
- 5 pts Pseudo-code provided, but algorithm is missing.
- 5 pts Incorrect/ Missing Runtime Analysis (Ans: $(|V| + |E|)\log |V|$)
- 5 pts Incorrect/ Missing Proof of Correctness
- 2 pts Incomplete proof of correctness
- 10 pts Major Error/ Mostly Incorrect/ Suboptimal Algorithm
- 20 pts Incorrect/ Missing answer

Problem 1

(40 points; 20 points each)

Suppose you want to find the shortest path from u to v in a graph G that *passes through* a specified vertex w . Design an algorithm that runs in $O((|V| + |E|) \log |V|)$ for each of the following settings. Justify its correctness and explain its runtime.

- a.) G is an undirected weighted graph.

Solution: In order to find the shortest path from u to v that goes through vertex w , we need to use Binary heap based Dijkstra's Algorithm which outputs the shortest path from the start to each node. We run Dijkstra's Algorithm using w as the starting point until. This will output the shortest path from w to all nodes. Then we need to flip the path from w to u to get the path from u to w . We can then concatenate the path from w to v to the end of the path from u to w to get the shortest path from u to v through w . Note: we will have to truncate the path from w to v to exclude the w , in order to get a continuous path; this makes the output u - w - v , not u - w - w - v which would be wrong. This works because there is no shorter way to get from w to u or from w to v because that's how Dijkstra's works, so the net path has to be the shortest path. The runtime is $O((|V| + |E|) \log |V|)$ because Dijkstra's algorithm runs in $O((|V| + |E|) \log |V|)$ time.

- b.) G is a directed weighted graph, and instead of a single u and v , we have k queries (u_1, v_1) , (u_2, v_2) , $\dots$, (u_k, v_k) where for each one we want to find *only the weight* of the shortest path that passes through w (which is fixed) where $w \in G$ and $0 < k \leq |V|$.

Solution: For this problem, we would run a version of Binary Heap based Dijkstra's algorithm which returns a dictionary containing nodes and their distance to the starting node. We first run this Dijkstra's starting at w , which will get output the distances to all potential v . Then we construct a reverse graph and run Dijkstra's again with w as the starting point which will output the distances to all potential u . From there, you can simply loop through the queries and use the dictionary to find the distance from w to u and add it to the distance from w to v to find the shortest distance from u to v through w . This works because Dijkstra's can be run on a weighted graph, but would only be able to find the nodes reachable from w which would be all the options for v because we know that there is a path from u to v through w . Similarly, once we reverse the graph, all the nodes reachable by w would be the potential starting points or u . This approach would give us the distances to all the nodes in the graph that can reach or be reached by w . This method is also $O((|V| + |E|) \log |V|)$ because we are running Dijkstra's algorithm at most a constant 2 times and Dijkstra's algorithm runs in $O((|V| + |E|) \log |V|)$ time and then we are simply looking through the dictionary which is more efficient and can be ignored.

1.2 b) 20 / 20

✓ - 0 pts *Correct*

- 3 pts Minor Error in Algorithm
- 5 pts Pseudo-code provided, but algorithm is missing.
- 5 pts Incorrect/ Missing Runtime Analysis
- 5 pts Incorrect/ Missing Proof of Correctness
- 10 pts Major Error/ Mostly Incorrect/ Suboptimal Algorithm
- 20 pts Incorrect/ Missing answer
- 7 pts Treated directed graph as undirected

Problem 2

(20 points)

A *feedback edge set* of an undirected graph $G = (V, E)$ is a subset $F \subseteq E$ such that *every* cycle in G contains at least one edge in F . Design an algorithm that takes as input a **connected** undirected weighted graph $G = (V, E)$ with all weights being positive and distinct, and returns a *minimum-weight* feedback edge set.

Justify correctness and running time, including explaining why your output is a feedback edge set (i.e. why every cycle contains edges on it), and why it is of minimum weight.

Hint: Think Kruskal's algorithm.

Solution: For this problem, we need to run a modified kruskal's algorithm which uses a flipped priority queue where it starts with the edges that has the highest weights. Additionally, when it finds an edge where u and v are already in the same cluster, it should add that edge to the feedback edge set. When Kruskal's finishes, the feedback edge set that it has returned will be the minimum-weight feedback edge set. This is because Kruskal's would be returning a maximum spanning tree which has a minimum feedback edge set as it's complement, so the edges not included in the Max spanning tree should be included in the minimum-weight feedback edge set. It is a feedback edge set because it only adds the edge to the set if u and v are in the same cluster which only happens if they have reached each other through other nodes which means the edge between them which you are adding to the feedback edge set is guaranteed to be a part of the cycle containing u and v . It is of minimum weight because Kruskal's is exploring the largest edges first, so the edge that completes the cycle (the one you are adding to the min-weight feedback edge set explained in the last sentence) would be the shortest edge in the loop.

The running time of this algorithm is $\mathcal{O}(E * \log(E))$ because that is the runtime of Kruskal's Algorithm and we are simply doing a slight variation of that.

2 Problem 2 20 / 20

✓ - 0 pts *Correct*

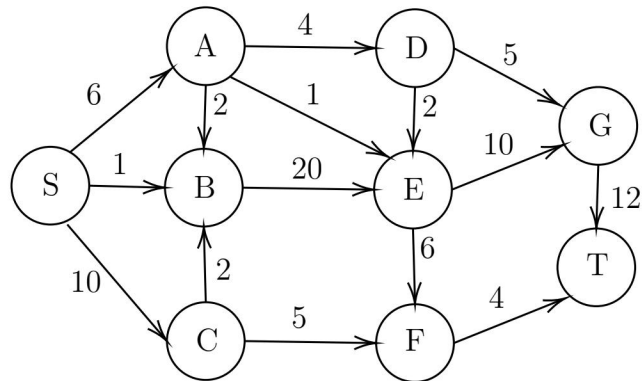
- 2 pts Minor Error in Algorithm
- 5 pts Incorrect/ Missing Runtime Analysis
- 2 pts Minor Error in Proof of Correctness
- 5 pts Incorrect/ Missing Proof of Correctness
- 10 pts Pseudo-code provided, but algorithm is missing.
- 10 pts Major Error/ Mostly Incorrect/ Suboptimal Algorithm
- 20 pts Incorrect/ Missing answer

💬 You should see it is feedback edge set because i

Problem 3

(20 points)

Consider the network shown below.



- a.) What is the maximum flow from S to T ? List the flow associated with each edge in the following format: (u, v, f_e) where $u \in G, v \in G$ and f_e is the flow along the edge $u \rightarrow v$.

Solution: The maximum flow is 13.

(S, A, 6)
(S, B, 1)
(S, C, 6)
(A, D, 4)
(A, B, 2)
(C, B, 2)
(C, F, 4)
(B, E, 5)
(D, G, 4)
(E, G, 5)
(F, T, 4)
(G, T, 9)

- b.) Find a matching minimum cut. Clearly list the edges in the minimum cut.

Solution: The minimum cut has a length of 13. The edges in the minimum cut are S-A, S-B, C-B, F-T.

3 Problem 3 20 / 20

✓ - 0 pts a) Correct

![Screen_Shot_2023-02-22_at_10.15.20_AM.png](/files/6c0efe05-a67b-4dfc-8a07-04f2674349c9)

![Screen_Shot_2023-02-22_at_11.58.06_AM.png](/files/bebe2d42-3682-409e-b95d-62d2b7a9121b)

- 2 pts a) One to three flows incorrect

- 5 pts a) Three to six flows incorrect

- 10 pts a) More than six flows incorrect

- 15 pts a) Missing answer

✓ - 0 pts b) Correct

![Screen_Shot_2023-02-22_at_12.00.45_PM.png](/files/e0a83cac-1b48-4c8b-ae42-208e1bee9bed)

- 1 pts b) One or two edges incorrect

- 3 pts b) More than two edges incorrect

- 5 pts b) Incorrect/ Missing Answer

Problem Set 4: Graph Algorithms

Prof. Abraham Ladha

Due: 02/20/2023 11:59pm

- Please TYPE your solutions using Latex or any other software. Handwritten solutions *won't* be accepted.
- Your solutions must be in plain English and mathematical expressions.
- Do not use pseudocode. Your answer will receive zero credit even if the pseudocode is correct.
- You can use the following algorithms as black-box for this problem set i.e. you don't have to explain the algorithm while using it. For example, you can write "Using DFS, compute the post labels":
 - BFS
 - DFS
 - Explore(v)
 - Toposort
 - SCC
 - Dijkstra's
 - Kruskal's
 - Construct a reverse graph
 - Construct the metagraph
- When we say "design an efficient algorithm", we are asking for a detailed description of the algorithm in English, i.e., without any code or pseudocode. Fastest (and correct) solutions are worth more credit.
- Show your work wherever necessary to receive full credits.